Ingegneria del Software — Soluzione del Tema 09/09/2024

Esercizio 1

Si consideri la seguente classe `TodoList` che mantiene un elenco di compiti (`Task`) da svolgere e permette di aggiungere nuovi compiti (metodo `add`) e indicare che un compito è stato completato (metodo `complete`). La classe mantiene i compiti ordinati secondo i seguenti criteri: (1) i compiti completati si trovano prima dei compiti ancora da completare, (2) i compiti completati sono ordinati in base al momento in cui sono stati completati, da quello completato meno recentemente a quello completato più recentemente, (3) i compiti non ancora completati sono ordinati in base al momento in cui sono stati aggiunti, da quello aggiunto meno recentemente a quello aggiunto più recentemente.

```
public class TodoList {
  // Ritorna il numero di compiti contenuti nella lista.
  public /*@ pure @*/ void size() { .. }

  // Ritorna il compito in posizione i-esima.
  public /*@ pure @*/ Task get(int i) { .. }

  // Aggiunge il compito alla lista (in fondo alla lista, essendo t il compito non
  // completato aggiunto piu' di recente).
  // Lancia una InvalidTaskException se il compito e' gia' completato.
  public void add(Task t) throws InvalidTaskException { .. }

  // Indica che il compito in posizione i e' stato completato.
  // Lancia una InvalidIndexException se la posizione non e' un indice valido per la lista
  // o se si riferisce a un compito gia' completato.
  public void complete(int i) throws InvalidIndexException { .. }
}

public class Task {
  public /*@ pure @*/ boolean isCompleted() { ... }
  ...
}
```

Nota: Si assume che il metodo `equals` della classe `Task` definisca l'uguaglianza tra due compiti senza tenere in considerazione il fatto che siano completati o meno.

Domanda a)

Si specifichi in JML il metodo `add`.

Soluzione

```
//@ requires t!=null;
//@
//@ ensures !t.isCompleted() && size()==\old(size())+1 &&
//@ (\forall int i; i>=0 && i<\old(size());
//@    get(i).equals(\old(get(i))) && (get(i).isCompleted() <==> \old(get(i)).isCompleted()) )  &&
//@ get(size()-1).equals(t);
//@
//@ signals (InvalidTaskException) t.isCompleted() && size()==\old(size()) &&
//@ (\forall int i; i>=0 && i<size();
//@    get(i).equals(\old(get(i))) && get(i).isCompleted() <==> \old(get(i).isCompleted()) );
public void add(Task t) throws InvalidTaskException { .. }
```

Domanda b)

Si specifichi in JML il metodo `complete`.

Soluzione

Quando si completa un compito, occorre mantenere i requisiti di ordinamento definiti dalla classe. Nella seguente definizione, si identifica un indice `j` che indica la posizione del primo compito non ancora completato prima dell'invocazione del metodo. Tutti compiti con un indice minore di `j` non vengono spostati, essendo compiti completati; il compito in posizione `i` viene spostato in posizione `j`; tutti i compiti dalla posizione `j+1` (inclusa) alla posizione `i` (esclusa) vengono spostati avanti di 1; tutti i compiti dalla posizione `i+1` (inclusa) in avanti, non vengono spostati.

```
//@ ensures i>=0 && i<\old(size()) && !\old(get(i).isCompleted()) &&
//@ size()==\old(size()) &&
//@ (\exists int j;
//@    (j>=0 && j<=i && !\old(get(j)).isCompleted() &&
//@       (\forall int w; w>=0 && w<j; \old(get(w)).isCompleted()) &&
```

```
//@    get(j).equals(\old(get(i))) && get(j).isCompleted() &&
//@       (\forall int w; w>=0 && w<j; get(w).equals(\old(get(w))) && get(w).isCompleted()) &&
//@       (\forall int w; w>j && w<=i; get(w).equals(\old(get(w-1))) && !get(w).isCompleted()) &&
//@       (\forall int w; w>i && w<size(); get(w).equals(\old(get(w))) && !get(w).isCompleted()));
//@
//@ signals (InvalidIndexException) (i<0 || i>=\old(size()) || \old(get(i)).isCompleted()) &&
//@ size()==\old(size()) &&
//@ (\forall int j; j>=0 && i<\old(size());
//@    get(j).equals(\old(get(j))) && get(j).isCompleted() <==> \old(get(j)).isCompleted() )
public void complete(int i) throws InvalidIndexException { .. }
```

Domanda c)

Si consideri un'implementazione della classe `TodoList` che fa uso di due liste per memorizzare rispettivamente i compiti completati e non completati, come mostrato in seguito.

```
public class TodoList {
  private List<Task> completedTasks;
  private List<Task> pendingTasks;
  ...
}
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione richiede che le due liste non siano nulle e non contengano elementi nulli. La lista `completedTasks` deve contenere solo compiti completati mentre la lista `pendingTasks` deve contenere solo compiti non completati.

```
//@ private invariant
//@ completedTasks!=null && pendingTasks!=null &&
//@ (\forall Task t; completedTasks.contains(t);
//@    (\forall Task t'; pendingTasks.contains(t');
//@       t!=null && t'!=null && t.isCompleted() &&!t'.isCompleted();
```

La funzione di astrazione deve definire i metodi pubblic `size` e `get` in funzione del contenuto delle due liste.

```
//@ private invariant
//@ size()==completedTasks.size()+pendingTasks.size() &&
//@ (\forall int i; i>=0 && i<completedTasks.size(); get(i)==completedTasks.get(i))&&
//@ (\forall int i; i>=0 && i<pendingTasks.size(); get(completedTasks.size()+i)==pendingTasks.get(i));
```

Domanda d)

Si consideri una classe `TodoList2` che aggiunge un metodo `addCompleted` che permette di aggiungere un compito completato alla lista (preservando tutti i criteri di ordinamentodefiniti in `TodoList`).

Può `TodoList2` essere definita come estensione di `TodoList` in accordo con il principio di sostituzione? Si motivi la propria risposta.

Soluzione

La classe `TodoList2` può essere definita come estensione di `TodoList`. La classe infatti aggiunge un metodo senza modificare i metodi esistenti e non viola alcuna proprietà della sopraclasse: si può osservare che un'invocazione del metodo `addCompleted` equivale a una invocazione di `add` seguita da un'invocazione di `completed(size()-1)`.

Esercizio 2

Si consideri la seguente implementazione di una classe `TodoList`, simile a quella presentata nel precedente esercizio. Si noti come unica differenza la presenza di un metodo `completeLast` per completare l'ultimo compito inserito, al posto del metodo `complete` presente nel precedente esercizio.

```
public class TodoList {
  private List<Task> completedTasks;
  private List<Task> pendingTasks;

  public TodoList() {
    completedTasks = new ArrayList<>();
    pendingTasks = new ArrayList<>();
  }

  public void add(Task task) {
    synchronized (pendingTasks) {
      pendingTasks.add(task);
    }
  }

  public void completeLast() {
    synchronized (pendingTasks) {
      Task t = pendingTasks.removeLast();
      t.complete();
      synchronized (completedTasks) {
        completedTasks.add(t);
      }
    }
  }

  public int size() {
    int size = 0;
    synchronized (completedTasks) {
      size += completedTasks.size();
    }
    synchronized (pendingTasks) {
      size += pendingTasks.size();
    }
    return size;
  }
}
```

Domanda a)

Nel suo insieme il codice evita il verificarsi di deadlock e di accessi concorrenti a zone condivise di memoria anche se i vari metodi fossero invocati da thread diversi? Motivare la propria risposta e in caso di risposta negativa spiegare cosa andrebbe cambiato per evitare deadlock o conflitti nell'accesso a memoria condivisa.

Soluzione

Sì, la sincronizzazione evita il verificarsi di deadlock e di accessi concorrenti a zone condivise di memoria. L'accesso a tutte le strutture dati condivise è protetto da blocchi `synchronized` sulle strutture dati stesse. E' presente un'unica sincronizzazione annidata che non può però dare luogo a deadlock.

Domanda b)

Si aggiunga alla classe `TodoList` il metodo `public Task get(int i)` descritto nell'esercizio 1, facendo particolare attenzione agli aspetti di sincronizzazione.

Soluzione

Dovendo, per fare i conti, accedere tanto a `pendingTasks` quanto a `completedTasks` ci si deve sincronizzare su entrambi gli oggetti e per farlo si deve seguire l'ordine di sincronizzazione adottato dal metodo `completeLast` onde evitare il rischio di deadlock.

```
public Task get(int i) {
    synchronized (pendingTasks) {
        synchronized (completedTasks) {
            if(i<completedTasks.size()) return completedTasks.get(i);
            else return pendingTasks.get(i-completedTasks.size());
        }
    }
```

```
}
```

Domanda c)

Si aggiunga alla classe `TodoList` il metodo `public Task getFirstPending()` che restituisce il primo compito non completato (pending) se presente, altrimenti sospende il chiamante in attesa che venga aggiunto un nuovo compito. Si chiarisca inoltre se e quali modifiche debbano essere fatte ai metodi esistenti.

Soluzione

```
public Task getFirstPending() {
    synchronized (pendingTasks) {
        while(pendingTasks.isEmpty())
            try { pendingTasks.wait(); }
            catch(InterruptedException ex) { ex.printStackTrace(); }
        return pendingTasks.get(0);
    }
}
```

Bisogna anche aggiungere l'istruzione `pendingTasks.notifyAll()` dentro il blocco sincronizzato del metodo `add`.

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java:

```
class Mammal {
  protected String color;
  public Mammal(String c) { color = c; }
  public String getColor() { return color; }
  public String speak(Mammal other) {
    return "I am "+getColor()+", it is "+other.getColor();
  }
}

class Feline extends Mammal  {
  public Feline(String c) { super(c); }
  public String getColor() { return color+" feline"; }
  public String speak(Feline other) {
    return "I am an aggressive "+getColor()+", it is "+other.getColor();
  }
}

class Cat extends Feline {
  public Cat(String c) { super(c); }
  public String  speak(Mammal other) {
    return "I am a cute "+getColor()+", it is "+other.getColor();
  }
  public String getColor() { return color+" cat"; }
}
```

e il seguente frammento di programma:

```
1  Mammal m1 = new Mammal("white");
2  Mammal m2 = new Feline("brown");
3  Mammal m3 = new Cat("striped");
4  Feline f1 = new Cat("spotted");
5  Cat c1 = new Cat("dark gray");
6  Cat c2 = new Feline("red");
7  System.out.println(m1.speak(m2));
8  System.out.println(m2.speak(m1));
9  System.out.println(f1.speak(c2));
10 System.out.println(m2.speak(m3));
11 System.out.println(m2.speak(f1));
12 System.out.println(f1.speak(m2));
13 System.out.println(c1.speak(m2));
14 System.out.println(c1.speak(f1));
```

Domanda a)

Ci sono linee del programma non valide? In caso affermativo indicarle, motivando brevemente la risposta e precisando se causano errori a tempo di compilazione o di esecuzione.

Soluzione

L'inizializzazione di `c2` alla linea 6 è errata e l'errore viene rilevato dal compilatore. Si elimina quindi anche la riga 9.

Domanda b)

Eliminate le righe che generano gli errori individuati al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta. Riportare sempre il numero di riga riportato nel testo.

Soluzione

7. I am white, it is brown feline
8. I am brown feline, it is white
10. I am brown feline, it is striped cat
11. I am brown feline, it is spotted cat
12. I am a cute spotted cat, it is brown feline
13. I am a cute dark gray cat, it is brown feline
14. I am an aggressive dark gray cat, it is spotted cat

Esercizio 4

Si consideri il seguente metodo:

```
1  public static int toBeTested (int x[], int g) {
2    int c=1;
3    int m=0;
4    int i=1;
5    if (x.length>1 && g>0) {
6      while (i<x.length && x[i]>0) {
7        if (x[i]>x[i-1] && x[i]<g) {
8          c++;
9          if (c > m) {
10           m=c;
11         }
12       } else {
13         c=1;
14       }
15       i++;
16     }
17   }
18   return m;
19 }
```

Domanda a)

Si determini un insieme *minimo* di test per la coppia (x, g) per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Basta un solo caso: per la coppia (x, g) prendere p.es. $(\{1, 2, 1\}, 100)$. L'insieme è ovviamente minimo.

Domanda b)

Si determini un insieme *minimo* di test per la coppia (x, g) per garantire la copertura delle decisioni (edge coverage).

Soluzione

Al caso precedente occorre aggiungere un caso per coprire il ramo nascosto del primo if, p.es., $(\{\}, -)$, dove il "-" sta per un valore qualunque.

Domanda c)

Si determini un insieme *minimo* di test per la coppia (x, g) per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Utilizziamo i due casi precedenti per coprire tutte le decisioni. Per coprire il caso falso della condizione $g > 0$ del primo if occorre aggiungere ad esempio il caso: $(\{1, 2\}, 0)$. Per coprire le condizioni del while e del secondo if occorre prendere anche: $(\{1, 2, 0\}, 1)$. La prima iterazione trova che $x[i] < g$ nel secondo if è falso ($x[1] < 1$); la seconda iterazione trova che $x[i] > 0$ nel while è falso ($x[2] > 0$).

Quindi bastano 4 casi. Non si possono ridurre ulteriormente.